NLP homework 2

Goryunov Kirill
Higher School of Economics, Faculty of Computer Science
(Dated: April 15, 2026)

The source code of the project can be viewed on github: https://github.com/randnull/nlp

I. INTRODUCTION

This work aims to research an N-gram algorighms.

II. DATASET

Our dataset for this task will be imdb-dataset-of-50k-movie-reviews, that contains more than 50.000 movie reviews for classification task and book War and Peace for text generation task.

III. TOKENIZATION

To tokenize text we will use regular expression:

```
(?:\w+')\w+|(?:[A-Z]\.)+|\w+(?:-\w+)*|[\w+\.]
```

IV. NGRAM

N-gram is an algorithm that creates groups of words of various lengths from text.

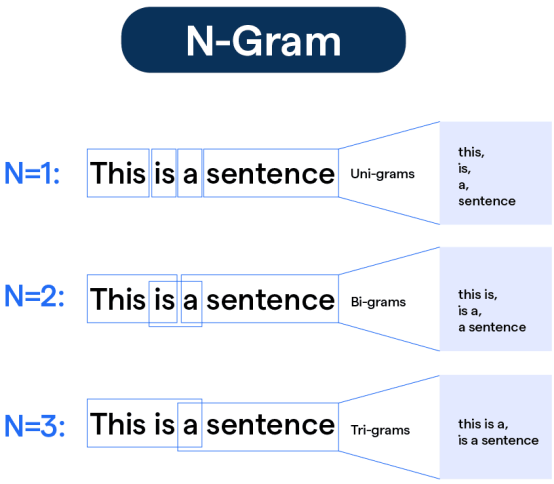

FIG. 1: N-gram

There are different type of ngram, based on length of group (1, 2, ...), called unigramm, bigram, ...

Also there are a several task, where we can use ngram algorightm.

A. Text generation

To solve the problem of text generation, we can compute a matrix based on the frequency with which words co-occur.

	Николаевич	быть	потому	и	,	так	не	аристократ	вижу	это	...	спины	собою	была	кокетства	победительно	красоту	красоты	необычайным	покат
<S>	1.0	1.0	6.0	3.0	1.0	3.0	6.0	4.0	2.0	3.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
и	0.0	1.0	0.0	5.0	14.0	1.0	6.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
не	0.0	1.0	0.0	5.0	3.0	0.0	4.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
быть	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
публика	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
победительно	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0
увлечить	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
порывистый	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
необычайным	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
плечами	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

FIG. 2: 2-gram

		Толстой	Война	и	мир	Первый	вариант	романа	От	автора	Я	...	красоту	жилала	красоты	чем-то	необычайным	покат	плечами	опустил	...
<S>	Лев	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Лев	Николаевич	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Николаевич	Толстой	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
графин	эмиссаром	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
и	и	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
чем-то	необычайным	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
необычайным	виконт	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
покат	плечами	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
перед	ним	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
все	тоже	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

FIG. 3: 3-gram

These matrix contains a lot of zeros cells. Adress this issue, we can apply Laplace-smoothed bigrams:

	Николаевич	быть	потому	и	,	так	не	аристократ	вижу	это	...	спины	собою	была	кокетства	победительно	кр
<S>	0.000887	0.000887	0.003103	0.001773	0.000887	0.001773	0.003103	0.002216	0.001330	0.001773	...	0.000443	0.000443	0.000443	0.000443	0.000443	0.0
и	0.000449	0.000898	0.000449	0.002695	0.006739	0.000898	0.003145	0.000449	0.000449	0.000449	...	0.000449	0.000449	0.000449	0.000449	0.000449	0.0
не	0.000473	0.000943	0.000473	0.002830	0.001887	0.000473	0.002358	0.000473	0.000473	0.000473	...	0.000473	0.000473	0.000473	0.000473	0.000473	0.0
быть	0.000494	0.000987	0.000494	0.000494	0.000987	0.000494	0.000494	0.000494	0.000494	0.000494	...	0.000494	0.000494	0.000494	0.000494	0.000494	0.0
публика	0.000496	0.000992	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	...	0.000496	0.000496	0.000496	0.000496	0.000496	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
победительно	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	...	0.000496	0.000496	0.000496	0.000496	0.000496	0.0
увлечить	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	...	0.000496	0.000496	0.000496	0.000496	0.000496	0.0
порывистый	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	...	0.000496	0.000496	0.000496	0.000496	0.000496	0.0
необычайным	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	...	0.000496	0.000496	0.000496	0.000496	0.000496	0.0
плечами	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	0.000496	...	0.000496	0.000496	0.000496	0.000496	0.000496	0.0

FIG. 4: 3-gram

Now we can take most propable next token for current token due to we use bigram:

$$P(w_i \mid w_1 w_2 \dots w_{i-1}) \approx P(w_i \mid w_{i-1})$$

To make generation not boring and not idempotent we will take sample from distibution of avaliable tokens, where our weights is data from dataframe after Laplase-smoothed.

Example of text generation we can see in fig 5.

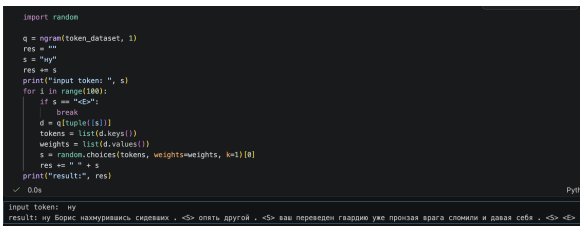

FIG. 5: Text generation

The text isn't very clear, but it is structured.

B. Classification

For classification task we will use logistic regression logistic regression 6.

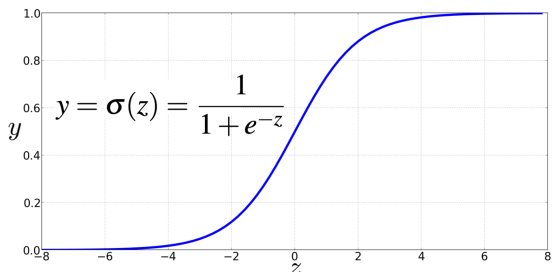

FIG. 6: Logistic regression

Our task is sentiment classification (positive/negative) and data is movies reviews. We will use 2 types of ngram: unigram: ("movie", "the", "interesting") and unigram + bigram: ("movie", "the", "interesting", "the_movie", "interesting_movie", ...)

Near we can see metric results for out classification model:

TABLE I: unigram lassification

class	precision	recall	f1-score	support
0	0.90	0.89	0.89	5000
1	0.89	0.90	0.89	5000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

TABLE II: unigram+bigram classification

class	precision	recall	f1-score	support
0	0.91	0.91	0.91	5000
1	0.91	0.91	0.91	5000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

Unigram+bigram model outperform unigram model for 1-2%. Both models show good accuracy. The ngram method is effective for the classification task.

V. RESULTS

N-grams are a powerful algorithm that can be applied to various types of tasks.